

2회차: 메뉴, 토스트 및 대화상자 (Dialog) 제어 기법 마스터 가이드

1. 안드로이드 메뉴(Menu) 시스템의 분류와 매커니즘

안드로이드의 메뉴는 사용 목적과 활성화 방식에 따라 크게 두 가지로 분류되며, 자바 코드와 XML 리소스 간의 연동 방식을 이해하는 것이 중요합니다.

- **옵션 메뉴 (Option Menu):** 액티비티 전체를 대표하는 표준 메뉴입니다. 일반적으로 화면 우측 상단의 액션바(Action Bar)에 위치한 '더보기(점 3개)' 버튼을 누르면 활성화됩니다.
- **컨텍스트 메뉴 (Context Menu):** 특정 위젯(Button, TextView, ListView 등)을 사용자가 길게 누르는 이벤트(Long Click)를 발생시켰을 때 독립된 팝업창 형태로 나타나는 메뉴입니다.

1.1 옵션 메뉴의 동작 원리 및 XML 구조

메뉴를 효율적으로 관리하기 위해 자바 코드에 직접 선언하기보다는 `res/menu/` 폴더 내에 XML 파일을 생성하여 정적으로 관리하는 것이 Best Practice입니다.

- `<menu>`: 메뉴의 최상위 루트 컨테이너 역할을 합니다.
- `<item>`: 개별 메뉴 항목을 정의합니다.
 - 주요 속성: `android:id` (이벤트 구별용 식별자), `android:title` (화면 표시 텍스트)
- `<menu>` 내부에 또 다른 `<menu>` 구조를 배치하면 서브 메뉴(Sub Menu)를 계층적으로 구성할 수 있습니다.

옵션 메뉴 구현 자바 핵심 분석 코드

Java

```
package com.exam.menu;

import android.graphics.Color;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.widget.LinearLayout;
import androidx.appcompat.app.AppCompatActivity;

public class MenuActivity extends AppCompatActivity{
```

```

private LinearLayout baseLayout;

@Override
protected void onCreate(Bundle savedInstanceState){
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_menu);
    baseLayout = findViewById(R.id.baseLayout);
}

/**
 * 1단계: 옵션 메뉴가 처음 생성될 때 시스템에 의해 단 한 번 호출되는 콜백 메서드
 * XML 리소스 파일을 메모리에 인플레이트(객체화)하여 액티비티의 액션바에 등록합니다.
 */
@Override
public boolean onCreateOptionsMenu(Menu menu){
    super.onCreateOptionsMenu(menu);

    // 메뉴 인플레이터(Inflater) 객체 확보
    MenuInflater mInflater = getMenuInflater();
    // res/menu/menu_option.xml 리소스를 전개하여 menu 객체에 삽입
    mInflater.inflate(R.menu.menu_option, menu);

    return true; // 반드시 true를 반환해야 화면에 메뉴가 정상 노출됨
}

/**
 * 2단계: 사용자가 활성화된 메뉴 아이템 중 하나를 터치했을 때 실행되는 이벤트 콜백 메서드
 */
@Override
public boolean onOptionsItemSelected(MenuItem item){
    // 클릭된 메뉴 아이템의 고유 ID를 판별하여 조건 분기 처리
    int itemId = item.getItemId();

    if (itemId == R.id.menu_bg_red) {
        baseLayout.setBackgroundColor(Color.RED);
        return true;
    } else if (itemId == R.id.menu_bg_blue) {
        baseLayout.setBackgroundColor(Color.BLUE);
        return true;
    } else if (itemId == R.id.menu_sub_rotate) {
        // 서브 메뉴 예시: 특정 레이아웃이나 위젯을 45도 회전
        baseLayout.setRotation(45);
        return true;
    }

    return super.onOptionsItemSelected(item);
}
}

/**
 * [실행 환경 및 시간 복잡도 분석]
 * - 실행 환경: Android 11+ (API 30) / Java 11 기반 표준 액티비티 구조
 * - 시간 복잡도: O(1). XML을 메모리에 부풀리는 작업 및 항목 아이디 인덱싱 분기는 모두 즉시(상수
시간) 처리됩니다.
 */

```

2. 토스트(Toast)의 깊이 있는 속성 제어

토스트는 화면 전면에 나타나 일시적으로 사용자에게 짧은 정보를 전달하고 자동으로 사라지는 간편형 메시지 뷰입니다.

- **기본 사용법:** `Toast.makeText(Context, "메시지 내용", Toast.LENGTH_SHORT).show();`
 -  **시험 포인트:** 마지막에 `.show()` 메서드를 누락하면 화면에 아무것도 출력되지 않으므로 코드 작성 시 주의해야 합니다.
- **위치 커스터마이징:** `setGravity(int gravity, int xOffset, int yOffset)` 메서드를 사용하면 기본 중앙 하단에 출력되는 토스트의 절대적인 좌표를 임의로 변경할 수 있습니다.
 - 예: 상단 우측에 띄우고 싶다면 `Gravity.TOP | Gravity.RIGHT` 속성과 간격 오프셋 값을 인자로 전달합니다.

3. 대화상자 (AlertDialog) 아키텍처 및 커스텀 제어 ★

대화상자는 사용자의 현재 작업을 일시 정지시키고 화면 중앙에 팝업을 띄워 최종 승인이나 데이터를 입력받도록 유도하는 필수 컴포넌트입니다.

3.1 알림 대화상자의 기본 구조

대화상자는 객체를 직접 생성하지 않고 안전한 프로그래밍 패턴인 `AlertDialog.Builder` 내부 클래스 빌더를 이용하여 속성을 체인 형태로 선언한 뒤 `.show()` 또는 `.create()` 로 빌드합니다.

- `setTitle()` : 대화상자 최상단 제목 텍스트 정의
- `setMessage()` : 본문 상세 안내 메시지 정의
- `setIcon()` : 알림창 테마 아이콘 정의 (`R.drawable` 또는 `R.mipmap` 리소스 연동)

3.2 대화상자 버튼의 종류와 이벤트 리스너 규칙

대화상자 하단에는 최대 3개의 버튼 유형을 배치하여 이벤트를 처리할 수 있습니다.

1. `setPositiveButton()` : 긍정적/확인 기능 수행
2. `setNegativeButton()` : 부정적/취소 기능 수행
3. `setNeutralButton()` : 중립적/보류 기능 수행

각 버튼의 두 번째 인자로 클릭 시 수행할 동작을 명시하기 위해

`DialogInterface.OnClickListener()` 익명 객체를 구현합니다.

3.3 복잡한 목록형 및 선택형 대화상자 요약

단순 텍스트 메시지 창 외에 여러 항목 중 하나를 고르게 유도하는 특수 대화상자 속성입니다.

대화상자 유형	사용하는 주요 메서드	화면 구성 요소 및 특징
목록 대화상자	<code>setItems(CharSequence[] items, ...)</code>	단순 문자열 배열 리스트가 수직으로 나열되며, 터치 시 즉시 이벤트 발생 후 창이 닫힘.
라디오 선택 창	<code>setSingleChoiceItems(..., int checkedItem, ...)</code>	항목 왼쪽에 라디오 버튼 이 동반되며, 오직 단 하나만 마킹 가능한 단일 선택 창 구성.
체크박스 다중 창	<code>setMultiChoiceItems(..., boolean[] checkedItems, ...)</code>	항목 우측에 체크박스 가 생성되어 여러 개의 옵션을 중복 선택(배열 상태 기록) 가능.

3.4 완벽 가이드: 레이아웃 인플레이터를 활용한 사용자 정의 (Custom) 대화상자 구현

시험 실습의 꽃이라 불리는 파트로, 개발자가 직접 디자인한 별도의 XML 파일을 불러와 대화상자 내부에 주입하고 데이터를 양방향으로 교환하는 고난도 기법입니다.

Java

```
// 1단계: 대화상자 내부에 들어갈 커스텀 뷰 객체를 레이아웃 인플레이터로 메모리에 전개
View dialogView = (View) View.inflate(MainActivity.this, R.layout.dialog_user_input, null);

// 2단계: 대화상자 빌더 선언 및 커스텀 뷰 세팅
AlertDialog.Builder dlg = new AlertDialog.Builder(MainActivity.this);
dlg.setTitle("사용자 정보 등록");
dlg.setIcon(R.drawable.ic_user_profile);
dlg.setView(dialogView); // [★핵심] 빌더 내부에 인플레이트된 뷰 주입

// 3단계: 긍정 버튼 클릭 시 커스텀 뷰 내부에 존재하는 에디트텍스트 데이터를 수집하는 로직 정의
dlg.setPositiveButton("확인", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialog, int which){
        // [★주의] 일반 findViewById가 아니라 dialogView 인스턴스를 통해 내부 위젯을 찾아야 함
        EditText dlgEdtName = dialogView.findViewById(R.id.dlgEdt1);
        EditText dlgEdtEmail = dialogView.findViewById(R.id.dlgEdt2);

        // 추출한 데이터를 메인 화면의 텍스트뷰(tvName, tvEmail)에 동적으로 매핑
        tvName.setText(dlgEdtName.getText().toString());
        tvEmail.setText(dlgEdtEmail.getText().toString());
    }
});

// 4단계: 취소 버튼 클릭 시 개발자 정의 커스텀 토스트(toast1.xml)로 알림 피드백 전송
dlg.setNegativeButton("취소", new DialogInterface.OnClickListener() {
```

```
@Override
public void onClick(DialogInterface dialog, int which){
    Toast toast = new Toast(MainActivity.this);
    View toastView = View.inflate(MainActivity.this, R.layout.toast1, null);
    TextView toastText = toastView.findViewById(R.id.toastText1);

    toastText.setText("정보 입력이 취소되었습니다.");
    toast.setView(toastView); // 커스텀 토스트 뷰 설정
    toast.show();
}
});
```